

第 23 章 文件系统、标签、UUID 与容量管理

从块设备签名到文件系统几何：把创建、身份、容量、增长与检查放进同一条证据链。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：块设备提供可寻址空间，文件系统管理其中的结构和身份；设备已经变大，不代表文件系统已经增长，已有签名也绝不代表可以直接格式化。

- 01识别真实对象确认设备路径、父子层级与当前用途
- 02读取签名与身份取得 TYPE、LABEL、UUID 和挂载证据
- 03建立类型模型区分 XFS 与 ext4 的工具和能力边界
- 04比较容量层次分别观察设备、文件系统、数据块和 inode
- 05选择最小操作创建、增长、缩小或检查只改变目标层
- 06完成分层验收分别验证容量、身份、结构和已有数据

专题地图

知识专题	从块空间到文件系统实例
知识专题	XFS 与 ext4 的工具和能力矩阵
操作专题	建立文件系统证据基线
操作专题	在确认空设备上创建文件系统
知识专题	LABEL、UUID 与身份生命周期
知识专题	块设备、文件系统、数据块与 inode 容量
操作专题	用 df 与 du 解释空间差异
操作专题	扩大 XFS 或 ext4
诊断专题	检查、修复与典型容量故障
经典任务	创建带标签 XFS；保留数据在线增长

阅读时持续回答

1. 当前命令作用于块设备、文件系统还是目录树？
2. 目标上是否已有需要保留的签名和数据？
3. TYPE、LABEL、UUID 分别证明什么？
4. 底层容量和文件系统容量是否已经同步？
5. XFS 与 ext4 的增长、缩小和检查工具怎样不同？
6. 操作后怎样独立证明身份、容量和数据都正确？

章节边界：分区表与设备签名归第 22 章；挂载、fstab 与 Swap 归第 24 章；LVM 容量池和 LV 生命周期归第 25 章。本章只在理解文件系统动作所必需时轻量引用它们。

第 23 章 · 正文

先判断命令改变哪一层，再讨论怎样执行

磁盘、分区或逻辑卷只提供一段可以寻址的块空间。真正让系统能够创建目录、记录文件名、分配数据块并追踪空闲空间的，是建立在块设备之上的文件系统。存储操作中最常见的误判，是把底层设备、文件系统实例、挂载关系和目录内容当成同一个对象：看到设备变大就以为文件系统已经扩容，看到 ``blkid`` 有签名就以为文件系统健康，或者看到容量异常就重新执行 ``mkfs``。

本章沿着“对象身份 → 类型与元数据 → 容量层次 → 类型专用操作 → 分层验证”推进。每次创建、增长、缩小或检查都必须回答：目标是谁，前置条件是否满足，操作会改变什么，哪些已有状态必须保留，以及哪一条证据才能证明最终结果。

目标块设备是谁？

上面是否已有签名？

文件系统是什么类型？

底层和文件系统各有多大？

操作后身份、容量和数据分别怎样验收？

概念

文件系统对象是建立在一个块设备上的独立数据组织实例，它拥有自己的类型、元数据、分配结构、标签和 UUID。块设备只是容器，挂载只是把这个实例接入目录树；因此 ``mkfs`` 创建的是新文件系统，而不是创建分区、扩大容量或修复挂载配置。

概念

标签（LABEL）是便于人阅读的文件系统属性，可表达用途，但不保证唯一。标签属于文件系统实例而不是挂载点；多个文件系统可以同名，所以写操作和持久引用不能只凭一个看起来正确的 LABEL 就忽略设备容量、父子层级和 UUID。

概念

UUID 用来区分当前文件系统实例，通常在创建时生成。它比 ``/dev/vdX`` 这类枚举路径更适合被外部配置引用，但并非物理磁盘永恒身份：重新格式化会产生新 UUID，块级克隆又可能复制 UUID，因此必须从当前目标读取真实值。

概念

块设备容量是分区、LV 或其他块设备当前向上提供的地址范围，是文件系统能够增长到哪里的外部上限。``lsblk`` 看到设备已经变大，只证明容器层发生了变化；文件系统是否接管新增空间，还要继续查看类型专用几何和 ``df``。

概念

文件系统容量是文件系统元数据当前能够管理的块和 inode 范围。``df`` 观察已挂载文件系统的总体分配状态，``du`` 遍历目录树并汇总可见文件占用；二者差异不是谁“不准”，而是观察对象不同。

概念

XFS 与 ext4 的调整边界决定了工具选择和风险。XFS 在 RHEL 9 的稳定路径中支持在线增长但不能原地缩小；ext4 可以增长，并可在卸载、检查和正确顺序下缩小。增长不是重新格式化，缩小也绝不是增长命令的反向调用。

操作语义先建立关键命令的接口地图

这里集中列出本章最重要的入口。正式专题会继续解释操作前提、输出、验证和安全边界。

lsblk -f / blkid

SYNOPSIS

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,LABEL,UUID,MOUNTPOINTS
blkid [-s TAG ...] DEVICE
```

先从全局层级识别候选设备，再对目标设备精确读取内容签名。二者能证明类型和身份线索，不能单独证明文件系统健康或设备可以安全格式化。

lsblk -f

快速查看 FSTYPE、LABEL、UUID 与挂载点。

-o ...

显式选择字段，避免把设备容量与文件系统身份混为一列。

blkid -s TYPE -s LABEL -s UUID DEVICE

只读取本章最关心的签名属性。

mkfs.xfs / mkfs.ext4

SYNOPSIS

```
mkfs.xfs [-L LABEL] DEVICE
mkfs.ext4 [-L LABEL] DEVICE
```

在命令最后指定的块设备上创建一个新的文件系统实例。该动作会写入新的元数据，必须建立在“无需保留原有内容”的明确判断上。

-L LABEL

在创建时设置文件系统标签。

DEVICE

真正被写入的块设备；路径必须在执行前重新核对。

已有签名

不是自动使用强制参数的理由，应先查清数据和归属。

xfs_info / tune2fs -l

SYNOPSIS

```
xfs_info MOUNT_POINT_OR_DEVICE
tune2fs -l DEVICE
```

读取文件系统类型专用的几何和元数据。它们回答文件系统当前怎样组织空间，不替代底层设备容量、挂载关系或业务数据检查。

`xfs_info`

查看 XFS 的数据区、块大小、allocation group 和格式特性。

`tune2fs -l`

只列出 ext 文件系统 superblock 信息，不执行修复。

`xfs_growfs`

SYNOPSIS

```
xfs_growfs [OPTIONS] MOUNT_POINT
```

让一个已经挂载的 XFS 接管底层设备新增的空间。默认不指定目标块数时，增长到当前底层可提供的最大范围。

`MOUNT_POINT`

作用对象的访问入口，不是任意父目录，也不是重新格式化用的设备路径。

`-D SIZE`

按文件系统块数设置目标数据区大小；普通扩容通常无需手工指定。

前提

底层设备已经变大，并且目标确实是已挂载 XFS。

`resize2fs`

SYNOPSIS

```
resize2fs DEVICE [NEW_SIZE]
```

调整 ext2/ext3/ext4 文件系统大小。省略目标大小时通常增长到容器可用边界；缩小与增长具有完全不同的前提和风险。

省略 `NEW_SIZE`

在底层已扩大后增长到可用最大范围。

`NEW_SIZE`

显式目标大小，必须确认单位和底层边界。

缩小

必须卸载并先执行 `e2fsck -f`，且文件系统必须先于底层容器缩小。

`fsck` / `xfs_repair` / `e2fsck`

SYNOPSIS

```
fscck -N DEVICE
xfs_repair -n DEVICE
e2fsck -n DEVICE
e2fsck -f DEVICE
```

检查和修复必须按文件系统类型选择工具，并以未挂载状态作为实际修复的基本边界。`fscck.xfs` 的成功退出不代表 XFS 已经完成检查。

`fscck -N`

只显示将调用的类型助手，不修改文件系统。

`-n`

只读收集结构问题证据。

`e2fsck -f`

强制完整检查；ext4 缩小前必须执行。

`xfs_repair -L`

可能清除日志并造成数据丢失，不是普通答案。

`df / du`

SYNOPSIS

```
df [-hT|-i|-B1] PATH
du [-shx|---apparent-size] PATH
```

`df` 读取包含目标路径的已挂载文件系统整体状态；`du` 遍历目录树并汇总它能看到的文件。两者只能回答各自层次的问题。

`df -hT`

用可读单位显示类型、总量、已用和可用。

`df -i`

查看 inode 容量，解释“还有字节却不能新建文件”。

`du -shx`

汇总目录并限制在同一文件系统。

`---apparent-size`

观察逻辑长度，而不是实际分配块。

[知识专题] 从块空间到文件系统实例：先分清命令作用层

面对一个存储目标时，最顺的切入点不是立即运行 `mkfs`，而是先画出对象链：底层块设备是谁、上面是否已有签名、文件系统是什么类型、当前管理多大空间、是否已经挂载。只要其中一层没有确认，写操作就可能落在错误对象上。

① [知识点] 块设备只提供地址范围，文件系统负责组织数据

块设备向上提供固定大小的数据块和读写接口。文件系统在这些块之上建立自己的元数据结构，用来回答：

- 哪些块空闲，哪些块已分配；
- 哪些目录项指向哪些 inode；
- 文件大小、权限、所有者和时间戳是什么；
- 崩溃后怎样通过日志或检查工具恢复一致状态；
- 标签、UUID 和类型等身份信息是什么。

因此，`lsblk` 显示设备存在，只能证明内核看到了块设备；它不能自动证明设备已经有文件系统，更不能证明文件系统健康。

② [知识点] `mkfs` 的作用对象是命令最后指定的块设备

`mkfs.xfs /dev/vdb1` 与 `mkfs.ext4 /dev/vdb1` 都直接对 `/dev/vdb1` 写入新的文件系统结构。目标可以是分区、整盘、LV 或其他块设备，命令并不会替你判断这个对象是否安全，也不会创建分区。

`mkfs` 不是：

- 分区工具；
- 文件系统检查工具；
- 扩容工具；
- 修复 UUID 引用错误的工具；
- 清理未知状态时的默认动作。

只在已经确认无需保留数据的新对象上运行 `mkfs`。如果 `blkid`、`lsblk -f`、挂载关系或上层归属显示目标正在使用，应停止写入并回到设备调查。

③ [知识点] 文件系统、挂载关系和目录内容是三个不同对象

文件系统可以存在但未挂载；目录可以存在但没有文件系统挂载在其上；同一个文件系统挂载后，目录树才成为访问入口。完整链路是：

```
块设备
→ 文件系统实例
→ 当前挂载关系
→ 目录树中的文件与目录
```

本章主要处理前两层，并在容量和 XFS 增长时轻量引用“已挂载”前提。挂载、`/etc/fstab` 和启动持久性归第 24 章《挂载、fstab、Swap 与启动持久性》。

④ [知识点] 同一个路径可以同时涉及多层容量

假设 `/srv/archive` 是一个挂载点：

- `lsblk` 看到提供存储的块设备大小；

- `xfs_info` 或 `tune2fs -l` 看到文件系统内部几何；
- `df -hT /srv/archive` 看到文件系统总量、已用和可用；
- `du -sh /srv/archive` 汇总目录树中可见文件占用。

当这些数字不一致时，不应先问“哪个命令错了”，而要先问“每条命令观察的是哪一层”。

⑤ [边界] 设备枚举名不是长期身份

`/dev/vdb1`、`/dev/sdb1` 等路径是当前内核枚举结果。它们适合描述当前操作对象，但不能凭练习环境记忆直接套用到另一台机器。写操作前至少重新核对：

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,LABEL,UUID,MOUNTPOINTS
blkid /dev/vdb1
```

发现设备名称、容量、父设备或已有签名与预期不一致时，停止操作。

[Cheatsheet] 块设备提供空间；文件系统组织空间；挂载提供目录入口；`mkfs` 创建新文件系统而不是扩容；路径相似不等于对象相同。

[知识专题] XFS 与 ext4：工具、增长、缩小和检查边界

XFS 与 ext4 都是 RHEL 9 常见的本地文件系统，但不能只把它们理解成 `mkfs` 后缀不同。类型决定了专用查询工具、增长入口、缩小能力、检查流程以及故障恢复方式。考试题要求哪一种类型，就按该类型建立和验证；已有系统则必须先查明类型再选择工具。

① [知识点] RHEL 9 默认本地文件系统语境偏向 XFS

RHEL 9 默认安装常使用 XFS。XFS 面向大容量和并发 I/O，支持在线增长；ext4 具有成熟的通用能力，既能增长，也能在严格离线条件下缩小。默认选择并不意味着可以忽略题目指定类型，也不意味着现有文件系统必然是 XFS。

判断类型应读取当前证据：

```
lsblk -f
blkid <DEVICE>
df -T <PATH>
```

② [知识点] 两类文件系统有不同的工具矩阵

目的	XFS	ext4
创建	<code>mkfs.xfs</code>	<code>mkfs.ext4</code> / <code>mke2fs -t ext4</code>
查询基本身份	<code>blkid</code> 、 <code>lsblk -f</code>	<code>blkid</code> 、 <code>lsblk -f</code>

目的	XFS	ext4
查询专用信息	<code>xfs_info</code>	<code>tune2fs -l</code> 、 <code>dumpe2fs -h</code>
修改 LABEL/UUID	<code>xfs_admin</code>	<code>tune2fs</code>
增长	<code>xfs_growfs</code>	<code>resize2fs</code>
缩小	RHEL 9 支持边界内不提供原地缩小	<code>resize2fs</code> ，必须离线
检查/修复	<code>xfs_repair</code>	<code>e2fsck</code>

`fsck` 是统一入口或分派器，不代表所有文件系统都采用同一检查算法。

③ [知识点] XFS 的稳定操作模型是在线增长、不可原地缩小

XFS 增长需要：

1. 底层块设备已经提供新增空间；
2. XFS 已挂载；
3. 对挂载点执行 `xfs_growfs`；
4. 使用 `df` 和 `xfs_info` 验证。

RHEL 9 的受支持管理路径中，XFS 不提供原地缩小。需要更小的 XFS 时，通常采用备份或迁移数据、重建较小文件系统、再恢复数据的方案。不要把上游工具可能出现的实验性选项当作 RHCSA 稳定答案。

④ [知识点] ext4 可以在线增长，但缩小必须离线

`resize2fs` 可以扩大未挂载 ext4，也可以在内核和文件系统支持时在线扩大已挂载 ext4。缩小 ext4 时必须先卸载并完成强制一致性检查：

```
卸载
→ e2fsck -f
→ resize2fs <DEVICE> <TARGET_SIZE>
→ 再缩小底层容器
```

如果还需要缩小分区或 LV，必须先让文件系统变得更小，再处理底层对象。反过来会截断文件系统仍可能使用的块。

⑤ [知识点] XFS 与 ext4 的崩溃恢复路径不同

XFS 在挂载时回放日志。`fsck.xfs` 是兼容占位程序，会成功退出但不执行传统意义的检查。真正的 XFS 元数据检查使用 `xfs_repair -n`，并应在文件系统未挂载时进行。

ext4 使用日志恢复和 `e2fsck`。执行实际检查或修复时同样以未挂载状态为基本边界。任何修复工具都只能尽力恢复文件系统结构一致性，不能保证业务文件内容完整。

[Cheatsheet] XFS: `xfs_info`、`xfs_growfs`、`xfs_repair`，只增长；ext4: `tune2fs -l`、`resize2fs`、`e2fsck`，可离线缩小。

[操作专题] 建立可信的文件系统证据基线

对任何创建、扩容、修改标签或检查操作，先保存变更前基线。最有效的调查不是堆积所有命令，而是让每条证据回答一个明确问题：对象是谁、是否已有签名、类型是什么、身份是什么、当前文件系统多大、是否已挂载。

① [操作] 使用 `lsblk -f` 建立全局拓扑与文件系统视图

作用对象：内核识别的块设备及其层级。

基本形式：

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,LABEL,UUID,MOUNTPOINTS
lsblk -f
```

重点字段：

- `PATH`：当前完整设备路径；
- `SIZE`：块设备层容量；
- `TYPE`：disk、part、lvm 等块设备类型；
- `FSTYPE`：识别到的内容类型；
- `LABEL`、`UUID`：文件系统身份；
- `MOUNTPOINTS`：当前挂载入口。

边界：`FSTYPE` 为空不能单独证明设备安全空闲。设备仍可能含有未识别数据、分区表、加密或其他上层元数据。

② [操作] 使用 `blkid` 精确读取目标设备签名

作用对象：指定块设备上的可识别内容签名。

```
blkid /dev/vdb1
blkid -s TYPE -s LABEL -s UUID /dev/vdb1
```

典型结果字段包括 `TYPE`、`LABEL`、`UUID`。这些值必须从当前目标读取，不能复制讲义或练习中的示例。

验证边界：`blkid` 识别到 XFS 或 ext4，只证明签名存在；它不证明文件系统已挂载、容量充足或元数据健康。

③ [操作] 使用 `xfs_info` 查询 XFS 几何

作用对象：XFS 文件系统实例。

```
xfs_info /srv/archive
# 对未挂载目标也可按本机手册支持形式使用设备路径
xfs_info /dev/vdb1
```

常见字段：

- `bsize`：文件系统块大小；
- `blocks`：数据区块数量；
- `agcount`：allocation group 数量；
- `sectsz`、`sunit`、`swidth`：扇区和条带相关几何；
- `reflink`、`crc` 等格式特性。

输出的具体默认值与设备大小、创建参数和 `xfsprogs` 版本有关，不应背诵某一份示例输出。

④ [操作] 使用 `tune2fs -l` 查询 ext4 superblock

作用对象：ext2/ext3/ext4 文件系统。

```
tune2fs -l /dev/vdb1
```

重点字段通常包括：

- `Filesystem volume name`；
- `Filesystem UUID`；
- `Filesystem state`；
- `Block count`、`Free blocks`；
- `Block size`；
- `Inode count`、`Free inodes`；
- `Last mounted on`、`Last mount time`。

`tune2fs -l` 是读取信息，不等同于 `e2fsck`。看到 `clean` 也不能替代需要时的离线检查。

⑤ [验证] 用两类入口交叉确认身份

创建或修改属性后，至少使用两个独立视图：

```
blkid /dev/vdb1
lsblk -f /dev/vdb1
```

对 XFS 可继续使用：

```
xfs_info /dev/vdb1
```

对 ext4 可继续使用：

```
tune2fs -l /dev/vdb1 | grep -E 'Filesystem volume name|Filesystem UUID|Block size|Block count'
```

如果两条命令仍显示旧 LABEL 或 UUID，先等待 udev 事件完成，再重新读取；不要立即重复修改。

[Cheatsheet] `lsblk -f` 看全局层级，`blkid` 看目标签名，`xfs_info` 看 XFS 几何，`tune2fs -l` 看 ext4 superbblock。

[操作专题] 在确认空设备上创建带标签的文件系统

创建文件系统是一条清晰但高风险的写路径：确认目标、排除已有使用、执行一次创建、立即读取真实身份。危险不在命令复杂，而在目标设备选错或把已有文件系统当成空设备。

① [操作] 创建前重新确认目标对象

假设题目指定候选设备 `/dev/vdb1`，创建前至少执行：

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,LABEL,UUID,MOUNTPOINTS /dev/vdb
blkid /dev/vdb1
findmnt -S /dev/vdb1
```

必要时继续检查它是否属于 LVM、RAID、加密或其他存储层。发现已有文件系统、挂载、业务数据或上层归属时，停止执行 `mkfs`，回到题意和相邻章节处理。

② [操作] 创建带 LABEL 的 XFS

```
mkfs.xfs -L ARCHIVE /dev/vdb1
```

- `-L ARCHIVE`：创建时设置文件系统 LABEL；
- `/dev/vdb1`：实际被写入的块设备；
- 默认格式特性由当前 RHEL 9 的 `xfsprogs` 和设备条件决定。

不要在已有 XFS 上重复运行该命令来“修复”容量或 UUID。创建动作会建立新的元数据结构。

③ [操作] 创建带 LABEL 的 ext4

```
mkfs.ext4 -L ARCHIVE /dev/vdb1
```

`mkfs.ext4` 通常是 `mke2fs` 的 ext4 前端。LABEL 有长度限制；ext4 标签最多 16 字节。对于包含多字节字符的标签，按字节限制理解，并在创建后读取真实结果。

④ [参数] 不把强制参数当作默认答案

当工具发现已有签名时可能拒绝继续或要求确认。正确反应不是立即添加 `-f` 或其他强制选项，而是重新调查：

```
已有签名是什么
→ 是否需要保留
→ 目标设备是否选错
→ 是否属于其他存储层
→ 题目是否明确要求重建
```

只有证据明确说明旧内容无需保留且题目要求重建时，才评估强制行为。本章经典任务不需要强制参数。

⑤ [验证] 创建后读取类型、标签和 UUID

```
blkid /dev/vdb1
lsblk -f /dev/vdb1
```

验证目标：

- `TYPE` 与题目指定类型一致；
- `LABEL` 为实际设置值；
- `UUID` 已由文件系统生成；
- 未影响相邻设备；
- 没有把挂载或 `fstab` 作为本章创建任务的隐含要求。

[Cheatsheet] 调查目标 → 确认无需保留 → `mkfs.<TYPE> -L <LABEL> <DEVICE>` → `blkid` + `lsblk -f`；强制格式化不是默认修复。

[知识专题] LABEL 与 UUID：文件系统身份怎样生成、修改和失效

文件系统身份位于文件系统元数据中。LABEL 适合人阅读和表达用途，UUID 适合稳定区分实例。两者都不是挂载点名称，也不是底层物理设备永远不变的属性。理解身份生命周期，可以避免为了修复错误引用而误格式化设备。

① [知识点] LABEL 可读，但不能假设唯一

管理员可以给多个文件系统设置相同 LABEL，因此 `LABEL=DATA` 只有在环境中保持唯一时才可靠。标签适合表达用途，例如 `ARCHIVE`、`BACKUP`，但不能因为标签符合预期就忽略设备容量、父设备和 UUID。

查询：

```
lsblk -f
blkid
```

② [知识点] UUID 标识的是当前文件系统实例

文件系统创建时通常生成 UUID。它比 `/dev/vdX` 枚举路径更适合被外部配置引用，但仍有两个边界：

- 重新格式化会创建新的文件系统和新的 UUID；
- 块级克隆可能复制 UUID，导致两个实例拥有相同身份，需要显式处理。

因此，UUID 必须来自当前目标，而不是来自旧截图、命令历史或另一台机器。

③ [操作] 修改 XFS 的 LABEL 或 UUID

XFS 属性修改以未挂载文件系统为稳定边界：

```
xfs_admin -L ARCHIVE2 /dev/vdb1
NEW_UUID=$(uuidgen)
xfs_admin -U "$NEW_UUID" /dev/vdb1
udevadm settle
```

只修改需要改变的属性。不要为了“刷新 UUID”无理由更改生产文件系统身份，因为外部配置可能引用旧值。

④ [操作] 修改 ext4 的 LABEL 或 UUID

```
tune2fs -L ARCHIVE2 /dev/vdb1
NEW_UUID=$(uuidgen)
tune2fs -U "$NEW_UUID" /dev/vdb1
udevadm settle
```

`tune2fs` 是 ext 系列文件系统属性管理工具。修改后用 `blkid` 和 `lsblk -f` 重新读取，不能只相信命令退出状态。

⑤ [诊断] 身份变化后先修正引用，不要再次格式化

症状示例：外部配置仍引用旧 UUID，而 `blkid` 显示新 UUID。调查链是：

```
读取当前 UUID
→ 确认文件系统和数据仍然存在
→ 找出引用旧 UUID 的配置
→ 最小修改引用
→ 再验证
```

完整的 `/etc/fstab` 修改和启动验收归第 24 章。本章只建立接口：属性变化会影响所有依赖 LABEL 或 UUID 的外部配置。

[Cheatsheet] LABEL 便于阅读但可能重复；UUID 属于文件系统实例；重建会换 UUID；修改属性后 `udevadm settle`，再用 `blkid` / `lsblk -f` 验证。

[知识专题] 容量的四层模型：块设备、文件系统、数据块和 inode

“空间不足”不是一个单一状态。底层设备可能还有空闲范围，但文件系统尚未增长；数据块可能还有余量，但 inode 已经耗尽；目录树汇总可能很小，但文件系统仍显示大量已用空间。只有先确定容量层，后续命令才有意义。

① [知识点] 块设备容量是文件系统能够增长的外部上限

`lsblk` 的 `SIZE` 描述当前块设备大小。分区或 LV 已扩大后，`lsblk` 可能立即显示新容量，但文件系统仍按旧几何管理原来的块范围。

```
lsblk -b -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS
```

看到底层变大只是“扩容前置条件完成”，不是文件系统扩容终态。

② [知识点] 文件系统几何决定当前管理范围

XFS 的数据区块数与块大小可以从 `xfs_info` 读取；ext4 的块数和块大小可从 `tune2fs -l` 读取。文件系统容量通常包含元数据、日志、保留块或其他内部开销，因此不应简单期待 `df` 的总量与设备字节数完全相等。

③ [知识点] `df` 报告已挂载文件系统的总体空间

```
df -hT /srv/archive  
df -B1 /srv/archive
```

常见字段：

- `Filesystem`：当前挂载源；
- `Type`：文件系统类型；
- `Size`、`Used`、`Avail`；
- `Use%`；
- `Mounted on`。

`df` 观察的是文件系统分配器视角，通常只能报告已挂载文件系统。它不告诉你某一个目录具体用了多少。

④ [知识点] inode 是独立于数据块的容量维度

大量小文件可能先耗尽 inode，而数据块仍有余量：

```
df -i /srv/archive
```

如果 `IUse%` 达到 100%，新文件创建会失败，即使 `df -h` 仍显示可用字节。删除不再需要的小文件、调整应用文件布局或迁移到适合的文件系统才是正确方向，不是简单扩大单个文件大小。

⑤ [知识点] 可用空间不等于业务可以写入

`df` 显示 `Avail` 大于零，也不能保证目标用户能写：目录权限、ACL、只读挂载、配额、SELinux、应用自身限制都可能阻止写入。这些问题分别归权限、挂载和安全章节。本章只说明容量证据不能扩大为完整业务可用性结论。

[Cheatsheet] `lsblk` 看容器，专用信息工具看文件系统几何，`df -h` 看数据块容量，`df -i` 看 inode；任一层有余量都不能替代其他层。

[操作专题] 用 `df` 与 `du` 回答不同的空间问题

`df` 和 `du` 经常被放在一起，却不是同一类统计。`df` 读取文件系统整体分配状态；`du` 遍历路径，累加它能够看到的文件所占块数。两者差异本身就是诊断证据，不应被当作“其中一个不准”。

① [操作] 用 `df -hT` 查看文件系统类型和整体容量

```
df -hT /srv/archive
```

将路径作为参数时，`df` 报告包含该路径的文件系统。使用 `-T` 显示类型，`-h` 采用可读单位；需要精确比较扩容前后字节数时可用 `-B1`。

边界：`df /srv/archive/subdir` 与 `df /srv/archive` 通常指向同一个文件系统，不代表两个目录各自拥有独立容量。

② [操作] 用 `du -sh` 汇总目录树实际分配空间

```
du -sh /srv/archive
du -shx /srv/archive/* 2>/dev/null
```

- `-s`：只输出汇总；
- `-h`：可读单位；
- `-x`：限制在同一文件系统，避免跨入子挂载。

`du` 需要遍历权限。无法读取的目录、被排除的挂载或竞态变化都会影响结果。

③ [参数] 逻辑大小与实际分配块不同

```
du -h sparse-file
du -h --apparent-size sparse-file
ls -lh sparse-file
```

稀疏文件的逻辑长度可能很大，但未实际分配同等数据块。默认 `du` 更接近已分配空间，`--apparent-size` 查看逻辑长度。判断存储压力时不要只看 `ls -l` 的文件大小。

④ [诊断] `df` 明显大于 `du` 时的高频原因

按区分度调查：

1. 已删除但仍被进程打开的文件：目录项消失，`du` 看不到，文件系统仍占块；使用 `lsof +L1` 或针对性进程证据检查。
2. 权限或遍历遗漏：当前用户无法读取部分目录；以合适权限重新统计。
3. 文件系统元数据、日志和 `ext4` 保留块：不会全部映射为普通文件。
4. 被挂载遮蔽的旧目录内容：旧数据仍占下层文件系统，但当前路径显示的是另一个挂载；完整处理归第 24 章。
5. 统计时数据持续变化：日志、数据库或缓存正在写入。

⑤ [诊断] `du` 结果大于预期时先定位目录，不立即删除

```
du -x -h --max-depth=1 /srv/archive | sort -h
du -x -a /srv/archive | sort -n | tail
```

先找出高占用路径，再识别文件所有者、用途、更新时间和是否仍被进程使用。不要把“删除最大的文件”当作通用答案；数据库、日志和应用文件可能需要应用级轮转或停写流程。

[Cheatsheet] `df` 看文件系统整体，`du` 看可遍历文件树；`-x` 不跨文件系统；`--apparent-size` 看逻辑长度；`df > du` 先查已删除但仍打开的文件。

[操作专题] 扩大已有文件系统：先扩容器，再让文件系统接管空间

文件系统增长不是创建新文件系统。正确的扩容链路必须保留现有身份和数据：先确认底层块设备已变大，再按文件系统类型执行增长，最后比较变更前后的容量和数据证据。

① [操作] 记录扩容前基线

假设 `/srv/archive` 是目标路径：

```
findmnt -no SOURCE,FSTYPE,TARGET /srv/archive
lsblk -b -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS
df -B1 -T /srv/archive
blkid <SOURCE_DEVICE>
sha256sum /srv/archive/marker.txt
```

本章只要求轻量使用 `findmnt` 确认当前源和类型；挂载配置本身归第 24 章。标记文件用于证明扩容没有通过重建文件系统完成。

② [判断] 底层设备必须先提供新增空间

比较：

```
lsblk 的设备 SIZE
与
df/xfs_info/tune2fs 的文件系统容量
```

若块设备大小没有变化，文件系统工具无法凭空获得空间，应回到第 22 章分区或第 25 章 LVM 处理。若块设备已变大而 `df` 仍是旧值，才进入文件系统增长。

③ [操作] 扩大已挂载 XFS

```
xfs_growfs /srv/archive
```

作用对象：挂载在 `/srv/archive` 的 XFS 文件系统。

默认不指定目标大小时，增长到当前底层设备能够提供的最大可用范围。目标参数是挂载点，不是随意选择的父目录。

增长后：

```
df -B1 -T /srv/archive
xfs_info /srv/archive
```

④ [操作] 扩大 ext4

底层设备已经增大后，`ext4` 可按状态选择：

```
# 已挂载，在线增长到容器可用最大范围
resize2fs /dev/vdb1

# 未挂载，也可执行增长
resize2fs /dev/vdb1
```

不指定大小时，`resize2fs` 通常增长到当前容器可用边界。若指定目标大小，必须明确单位并确认底层能够容纳。

⑤ [验证] 容量、身份和数据必须分别验收

```
lsblk -b -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS
df -B1 -T /srv/archive
blkid <SOURCE_DEVICE>
sha256sum /srv/archive/marker.txt
```

验收含义：

- `lsblk`：底层容量；
- `df`：文件系统已接管新增容量；
- `blkid`：类型、LABEL、UUID 未被无意重建；
- 校验值：已有文件内容未改变。

⑥ [边界] 命令成功不能替代目标容量判断

`xfs_growfs` 或 `resize2fs` 无报错，只能证明工具没有报告失败。还需确认：

- 增加的容量是否符合题意；
- 操作对象是否正确；
- `df` 是否变化；
- 数据是否保留；
- 文件系统是否仍为指定类型。

[Cheatsheet] 基线 → 底层已扩展 → 按类型增长 → `lsblk` + `df` + 专用信息工具 → UUID/LABEL 和标记文件不变。

[知识专题] 缩小不是增长的反向命令

缩小文件系统需要把仍在使用的数据和元数据移动到新的边界以内，并保证底层容器不会截断有效块。它的风险远高于增长。RHCSA 中最重要的不是记住复杂缩容技巧，而是准确判断哪些文件系统根本不能缩小、`ext4` 缩小时顺序为何不可颠倒。

① [边界] XFS 在 RHEL 9 支持路径中不能原地缩小

不存在与 `xfs_growfs` 对称的稳定 RHCSA 缩小命令。需要更小 XFS 时，采用：

```
备份或迁移数据
→ 创建较小的新 XFS
→ 恢复数据
→ 分层验证
```

直接缩小其分区或 LV 会截断文件系统，造成损坏。

② [边界] ext4 缩小必须卸载

在线 ext4 可以增长，但不能在线缩小。标准顺序：

```
umount <MOUNT_POINT>
e2fsck -f <DEVICE>
resize2fs <DEVICE> <TARGET_SIZE>
```

卸载是否安全、哪个进程占用挂载点、如何修改底层分区或 LV，分别需要相邻章节的状态证据。

③ [操作语义] 文件系统必须先于底层容器缩小

正确顺序：

```
确认备份和停机窗口
→ 卸载
→ 检查文件系统
→ 缩小文件系统
→ 缩小分区或 LV
→ 重新接入并验证
```

错误顺序 先 lvreduce/缩分区 → 再 resize2fs 会让底层边界先变小，文件系统仍可能引用被截断的块。

④ [判断] 目标大小必须容纳现有数据和元数据

du 看到的数据量不是安全最小值。文件系统还需要 inode 表、日志、块组、目录结构和空闲空间。

resize2fs -P <DEVICE> 可以估算 ext 文件系统最小块数，但估算值仍不应被理解为无需备份的保证。

⑤ [安全] 缩小前必须具备可恢复路径

因为缩小涉及离线、数据移动和多层边界，生产变更至少应具备：

- 已验证备份；
- 明确回滚或重建方案；
- 停机窗口；
- 文件系统和底层设备的变更前记录；
- 缩小后重新挂载、容量、数据和应用验证。

本章只覆盖文件系统层必要范围，不把复杂缩容作为普通考试创建题的默认答案。

[Cheatsheet] XFS 不原地缩小；ext4 必须离线，先 e2fsck -f，先缩文件系统再缩底层；缩小前必须有恢复路径。

[诊断专题] 文件系统检查与修复：一致性、日志回放和数据恢复不是一回事

检查工具处理的是文件系统结构。它们可能重新连接孤立 inode、修复分配记录或丢弃无法恢复的元数据，但不能保证每个业务文件内容正确。诊断时先判断类型、挂载状态和故障性质，再选择只读检查或实际修复。

① [知识点] `fsck` 是前端，实际行为由文件系统类型决定

```
fsck -N /dev/vdb1
```

`-N` 只显示计划执行的命令，可用于理解分派。不要在未知类型或仍挂载的文件系统上直接运行实际 `fsck`。不同类型可能调用不同助手，XFS 更不能按 `ext4` 流程理解。

② [边界] `fsck.xfs` 成功退出不代表 XFS 已检查

RHEL 9 中 `fsck.xfs` 主要为兼容启动脚本存在，会立即返回成功。以下结论是错误的：

```
fsck.xfs 返回 0
→ XFS 元数据已扫描且健康
```

XFS 的正常崩溃恢复首先依靠挂载时日志回放；需要离线检查时使用 `xfs_repair -n`。

③ [操作] 对 XFS 执行只读检查

在确认文件系统未挂载后：

```
xfs_repair -n /dev/vdb1
```

`-n` 表示不修改文件系统，用于收集结构问题证据。若文件系统带有需要回放的脏日志，正常路径通常是先在安全条件下挂载并卸载以回放日志，再进行检查。

④ [操作] 实际修复 XFS 的风险边界

```
xfs_repair /dev/vdb1
```

只在未挂载、已有备份或恢复计划、并确认需要修复时执行。`xfs_repair -L` 会清除日志，只在日志无法回放且已接受潜在数据丢失时作为最后手段，不是普通答案。

⑤ [操作] 对 `ext4` 执行检查

只读检查：

```
e2fsck -n /dev/vdb1
```

强制完整检查并允许交互修复：

```
e2fsck -f /dev/vdb1
```

自动回答“是”的选项可能造成不可逆修改，不应在没有备份和输出审阅时作为默认方式。执行实际检查和修复时以未挂载状态为基本边界。

⑥ [验证] 修复后验证结构、身份和数据

修复命令退出后继续：

```
blkid /dev/vdb1
lsblk -f /dev/vdb1
# 按类型读取几何
xfs_info /dev/vdb1
# 或
tune2fs -l /dev/vdb1
```

随后在第 24 章的挂载流程中重新接入，检查日志、关键目录和业务文件。修复工具没有报错不能证明业务数据完整。

[Cheatsheet] 先识别类型和挂载状态；`fsck.xfs` 不做真实检查；XFS 用 `xfs_repair -n`，`ext4` 用 `e2fsck -n/-f`；实际修复前必须有恢复路径。

[诊断专题] 根据症状定位容量、身份或一致性层

稳定诊断链是：症状 → 当前证据 → 假设 → 下一条最有区分度的证据 → 最小修复 → 再验证。文件系统问题尤其不能以重新格式化作为通用起点，因为这会把可诊断问题变成数据丢失。

① [诊断] `lsblk` 已变大，但 `df` 仍是旧容量

当前证据：底层块设备容量已经增加，文件系统视图没有变化。

假设：文件系统尚未增长，或使用了错误类型的增长工具。

下一条证据：

```
findmnt -no SOURCE,FSTYPE,TARGET <PATH>
lsblk -b -o NAME,PATH,SIZE,FSTYPE,MOUNTPOINTS
df -B1 -T <PATH>
```

最小修复：XFS 用 `xfs_growfs <MOUNT_POINT>`；`ext4` 用 `resize2fs <DEVICE>`。

再验证：`df`、专用信息工具、UUID 和标记文件。

② [诊断] `blkid` 有签名，但文件系统无法正常接入

`blkid` 只证明存在可识别签名。继续检查：

- 类型是否正确
 - 设备是否为预期对象
 - 是否已经挂载在其他位置
 - 内核或系统日志有什么错误
 - 是否需要离线只读检查

不要因为挂载失败而执行 `mkfs`。那会覆盖原文件系统，使后续修复和取证更困难。

③ [诊断] `df` 使用率很高，但 `du` 汇总较小

优先调查：

```
du -shx <MOUNT_POINT>
ls -l +L1
```

若发现已删除但仍打开的大文件，应确认所属进程和应用的安全释放方式。直接重启整机或杀进程不是第一答案；先判断服务管理关系和业务影响。

④ [诊断] 数据块仍有空间，但无法创建新文件

检查 inode：

```
df -h <PATH>
df -i <PATH>
```

若 `IUse%` 为 100%，问题是 inode 耗尽。下一步定位产生大量小文件的目录和应用，而不是只扩大单个文件或把 `df -h` 的可用字节当作反证。

⑤ [诊断] 重新格式化后旧 UUID 引用失效

事实：`mkfs` 创建了新文件系统实例，旧 UUID 不再对应当前签名。

正确路径：

```
blkid <DEVICE>
lsblk -f <DEVICE>
```

确认当前 UUID，再修正外部引用。不要为了让 UUID“匹配旧配置”继续反复格式化。完整持久挂载修复归第 24 章。

[Cheatsheet] 设备大而 `df` 小：文件系统未增长；签名存在但不可用：查类型/日志/检查；`df > du`：查打开的已删除文件；能用块但不能建文件：查 inode；UUID 错：修引用，不重建。

[经典任务] 在确认空设备上创建带标签的 XFS

服务器提供候选块设备 `/dev/vdb1`。该名称只用于本任务，真实环境必须重新识别设备。要求在确认该设备没有需要保留的数据、没有当前挂载且不属于其他存储用途后，在其上创建 XFS，LABEL 设置为 `ARCHIVE`。

完成后必须从当前文件系统读取真实 UUID，并用至少两条独立命令证明：

- 文件系统类型为 XFS；
- LABEL 为 `ARCHIVE`；
- UUID 已实际生成；
- 没有把相邻设备作为操作对象。

限制条件：

- 不使用 `wipefs -a`；
- 不使用无调查的强制格式化；
- 不复制示例 UUID；
- 发现已有文件系统、挂载、LVM/RAID/加密归属或数据时停止写操作；
- 本任务不要求挂载或配置 `/etc/fstab`。

验收证据应包含调查命令、实际创建命令和创建后查询。不能只提交 `mkfs.xfs` 一条命令。

请先独立完成。参考解答从下一页开始。

[参考解答] 从设备调查到文件系统身份验证

① [调查] 重新识别候选对象

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,LABEL,UUID,MOUNTPOINTS /dev/vdb
blkid /dev/vdb1
findmnt -S /dev/vdb1
```

若命令显示目标已有文件系统、挂载点或与题意不符的容量和父设备，停止继续。必要时由相邻章节的方法检查 LVM、RAID 或加密归属。本解答不假设 `blkid` 无输出就一定安全。

② [判断] 明确创建条件

只有在以下条件同时成立时继续：

设备路径与容量符合题意
且
没有需要保留的文件系统或数据
且
没有当前挂载
且
没有其他上层存储归属

任何一项不满足，都不应通过强制参数掩盖。

③ [操作] 创建带标签的 XFS

```
mkfs.xfs -L ARCHIVE /dev/vdb1
```

该命令直接在 `/dev/vdb1` 上建立新 XFS。`ARCHIVE` 是文件系统 LABEL，不是挂载点或设备名。

④ [验证] 交叉读取真实身份

```
blkid /dev/vdb1  
lsblk -f /dev/vdb1  
xfs_info /dev/vdb1
```

判断标准：

- `TYPE="xfs"`；
- `LABEL="ARCHIVE"`；
- UUID 为当前命令实际读取到的值；
- `xfs_info` 能识别 XFS 几何；
- `/dev/vdb` 上其他对象未被改变。

⑤ [典型错误]

- 直接对 `/dev/vdb` 而不是题目指定的 `/dev/vdb1` 格式化；
- 因 `blkid` 无输出就省略其他归属调查；
- 复制示例 UUID 写进答案；
- 发现旧签名后直接加 `-f`；
- 为了验证而提前进入挂载和 `fstab`，导致章节层次混乱。

[Cheatsheet] 身份/签名/挂载调查 → 明确无需保留 → `mkfs.xfs -L ARCHIVE` → `blkid` + `lsblk -f` + `xfs_info`。

[经典任务] 扩展已有 XFS，并证明数据与身份未被重建

`/srv/archive` 当前是一个已挂载的 XFS 文件系统。基础设施操作已经扩大其底层块设备，但文件系统仍显示旧容量。目录中存在标记文件 `/srv/archive/marker.txt`，该文件必须保留。

要求：

1. 确认当前挂载源和文件系统类型；
2. 证明底层块设备容量已经大于文件系统当前容量；
3. 使 XFS 接管全部新增空间；
4. 证明扩容后 UUID、LABEL 和标记文件内容没有被无意改变；
5. 给出分层验收证据。

限制条件：

- 不重新执行 `mkfs.xfs`；
- 不修改分区或 LVM；
- 不删除标记文件；
- 不把命令退出状态或“显示 active”之类局部证据扩大为终态正确；
- 不编造设备名、旧容量、新容量或 UUID。

请先独立完成。参考解答从下一页开始。

[参考解答] 用容量层次和数据校验完成 XFS 在线增长

① [调查] 保存扩容前基线

```
findmnt -no SOURCE,FSTYPE,TARGET /srv/archive
SOURCE=$(findmnt -no SOURCE /srv/archive)
lsblk -b -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS
df -B1 -T /srv/archive
blkid "$SOURCE"
xfs_info /srv/archive
sha256sum /srv/archive/marker.txt
```

记录真实 `SOURCE`、类型、设备容量、文件系统容量、LABEL、UUID 和校验值。若 `FSTYPE` 不是 XFS，停止并选择对应文件系统流程。

② [判断] 区分底层容量与文件系统容量

`lsblk` 应显示目标源所在块设备已经扩大，而 `df -B1` 和 `xfs_info` 仍反映旧文件系统范围。若底层容量没有增加，本章没有可执行的文件系统增长动作，应回到分区或 LVM 层。

③ [操作] 对挂载点增长 XFS

```
xfs_growfs /srv/archive
```

不指定 `-D` 目标块数时，XFS 使用当前底层可提供的最大数据区范围。命令作用于挂载在 `/srv/archive` 的 XFS，而不是创建新文件系统。

④ [验证] 比较变更后容量

```
lsblk -b -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS  
df -B1 -T /srv/archive  
xfs_info /srv/archive
```

`df` 的文件系统总容量应增加，且仍为 XFS。若没有变化，重新核对挂载源、底层设备是否真的扩大以及命令是否作用于正确挂载点。

⑤ [验证] 身份与数据保留

```
SOURCE=$(findmnt -no SOURCE /srv/archive)  
blkid "$SOURCE"  
sha256sum /srv/archive/marker.txt
```

与基线比较：

- UUID 未因重建而变化；
- LABEL 未被无意修改；
- 标记文件校验值一致；
- 文件系统容量已经增加。

⑥ [典型错误]

- 对底层设备执行 `mkfs.xfs`，把扩容变成重建；
- 把 `xfs_growfs` 的目标写成错误目录或未挂载设备；
- 只看 `lsblk` 变大就宣布完成；
- 只看 `df` 变大，不验证身份和数据；
- 底层尚未扩展时反复执行增长命令。

[Cheatsheet] 保存源/类型/容量/UUID/校验值 → 确认底层大于文件系统 → `xfs_growfs <MOUNT_POINT>` → `df / xfs_info` → UUID、LABEL 和数据校验不变。

[知识专题] 工作迁移：把考试命令变成可审计的存储变更

真实工作中的文件系统通常承载已有业务，而不是空白练习设备。安全变更应把“能执行命令”升级为“能说明对象、风险、证据和回滚”。

① [工作方法] 变更前保存对象快照

至少记录：

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,LABEL,UUID,MOUNTPOINTS
blkid
findmnt
# 针对目标文件系统
df -B1 -T <PATH>
df -i <PATH>
```

还应记录应用停写状态、备份时间和关键文件校验值。

② [工作方法] 一次只推进一个存储层

把变更拆成：

```
底层容量变更
→ 文件系统增长
→ 挂载/持久配置
→ 应用功能验证
```

每层完成后立即验证，不把多个不可逆操作串成一条无法定位失败点的脚本。

③ [工作方法] 自动化前先建立幂等状态模型

后续使用 Ansible 自动化时，必须明确：

- 目标设备怎样稳定识别；
- 文件系统已存在时是否允许重建；
- 类型、LABEL、UUID 和大小的期望状态；
- 增长是否允许、缩小是否禁止；
- 操作后的验证和失败保护。

自动化模块不是绕过调查和破坏性边界的理由。本章只提供手工对象模型，RHCE 存储自动化另章展开。

[Cheatsheet] 变更前基线、一次推进一层、保留恢复路径、用独立证据验收、自动化也不得默认重建已有文件系统。

[本章收束] 把每次文件系统变更还原成对象、条件和证据

文件系统管理最危险的误判，是只看见一个设备路径或一条成功退出的命令，就把它扩大成“存储已经正确”。稳定方法始终从对象层次开始：先确认真正的块设备，再读取文件系统类型和身份，然后比较底层容量与文件系统容量，最后才选择创建、增长、缩小或检查工具。

一条可执行的工作方法

识别真实对象

→ 保存 TYPE、LABEL、UUID 与容量基线

→ 判断已有签名是否需要保留

→ 按 XFS/ext4 选择专用工具

→ 一次只改变一个存储层

→ 分别验证容量、身份与数据

任何写操作前都应能回答：命令最后作用于哪个设备；该设备是否已有需要保留的文件系统；文件系统当前是否挂载；底层容量是否已经满足前提；失败时是否有恢复路径。答不清这些问题，就不进入 `mkfs`、修复或缩小操作。

章末检查清单

- 能把块设备容量、文件系统容量、`df` 使用量和 `du` 汇总量分开解释；
- 能用 `lsblk -f` 与 `blkid` 交叉识别 TYPE、LABEL 和 UUID；
- 知道 `mkfs` 创建新实例，不能用于扩容或修复引用；
- 能按类型选择 `xfs_info`、`tune2fs -l`、`xfs_growfs`、`resize2fs`、`xfs_repair` 或 `e2fsck`；
- 能说明 XFS 不可原地缩小，`ext4` 缩小必须离线并先检查；
- 能解释每条验证命令证明什么，以及它不能证明什么；
- 面对已有签名、挂载失败或容量不一致时，先取得下一条有区分度的证据，而不是重新格式化。

主要判断表

看到的现象	首先说明什么	下一条证据	不应直接做什么
<code>lsblk</code> 容量已增大， <code>df</code> 仍是旧值	底层与文件系统容量不同步	读取 FSTYPE、挂载点和专用几何	重新运行 <code>mkfs</code>
<code>blkid</code> 能看到 TYPE/UUID	设备上存在可识别签名	查挂载状态、日志与只读检查结果	把签名存在等同于健康
<code>df</code> 高、 <code>du</code> 低	整体分配与可遍历目录树不一致	<code>lsdf +L1</code> 、权限和子挂载证据	盲目删除最大目录
<code>df -h</code> 有空间但不能新建文件	可能不是数据块容量问题	<code>df -i</code>	只扩大单个文件
题目要求缩小 XFS	当前类型不支持原地缩小	规划迁移、重建和恢复	缩小分区或 LV 截断 XFS

看到的现象	首先说明什么	下一条证据	不应直接做什么
已有签名但用途不明确	破坏性前提未满足	查设备归属、挂载与数据责任人	使用 <code>-f</code> 或 <code>wipefs -a</code> 掩盖风险

向下一章交接

本章已经建立了文件系统自身的类型、身份、几何和容量状态。第 24 章《挂载、fstab、Swap 与启动持久性》将在此基础上继续回答：怎样把文件系统接入目录树；怎样用 UUID 或 LABEL 建立持久引用；怎样区分当前挂载、配置状态与重启后的真实终态。本章不提前展开完整的 `mount`、`findmnt`、`/etc/fstab` 和 Swap 配置流程。